

Playwright Interview Preparation Handbook (2026 Edition)

Volume 2 - Enterprise Framework, CI/CD, Performance, Real Coding Challenges, QA Architect Scenarios, Broken-Code Debugging

Prepared for Srikanth, SDET and Associate Architect QE

September 2026

Contents

How to Use This Volume	3
SECTION 11 — Enterprise Framework & POM (Real Scenarios)	4
Q1. Your team inherited a 600-test Playwright suite where every Page Object method also contains expect () calls. A new hire's PR adds a test that fails with a confusing stack trace three layers deep inside a Page Object. How do you fix the architecture, not just this one test?	4
Q2. Four engineers are now writing Playwright tests in parallel on the same feature area, and every PR has merge conflicts in one giant DashboardPage.ts file. How do you restructure this without blocking the team for a week?	5
Q3. A Page Object method works fine for the web app, but the mobile-responsive layout renders the same feature with a completely different DOM structure. Do you fork the Page Object, or is there a better pattern?	6
Q4. Your test-data folder has grown into 80 loosely-related JSON files, and nobody's sure which ones are still used. How do you clean this up without breaking the suite?	7
SECTION 12 — CI/CD (Real Pipeline Incidents)	7
Q5. Your GitHub Actions pipeline passed on your feature branch but fails on main after merge, with no code difference in the failing test. What's your triage process?	7
Q6. Your 800-test suite takes 45 minutes in CI, and the team wants PR feedback under 10 minutes. Walk through exactly how you'd get there.	8
Q7. A flaky test has a 5% failure rate and is now auto-retried twice in CI, effectively hiding the flakiness from the team. How do you surface and fix it without just removing the retry and blocking every PR?	9
Q8. Your Docker-based Playwright pipeline works on your laptop's Docker but fails with browser launch errors only inside the CI runner's container. Diagnose it.	10
SECTION 13 — Performance, Stability & Optimization (Real Tuning Decisions)	11
Q9. After adding more CI workers, your suite got slower, not faster. Explain what's actually happening and how you'd right-size it.	11
Q10. Your suite's memory usage climbs steadily over a 3-hour nightly run and eventually OOM-kills the CI job partway through. Find the leak.	11
Q11. A specific test suite runs fine in isolation but consistently times out only when run as the last file in a full 800-test run. What's your hypothesis?	12
SECTION 14 — Real Coding Interview Questions (Live Coding Style)	13

Q12. Build a fixture that logs in as one of three roles (admin, analyst, auditor), reusing cached auth per worker, and expose it as <code>test.use({ role: 'admin' })</code> per describe block.	13
Q13. This test intermittently fails with “element is not attached to the DOM.” Find the bug and fix it.	15
Q14. Automate uploading a file via a custom drag-and-drop zone that doesn’t respond to <code>setInputFiles()</code> because there’s no visible <code><input type="file"></code>	16
Q15. Write a test that verifies a table’s data is correctly sorted after clicking a column header, for a column containing mixed numeric-looking strings (e.g., patient IDs like “P-102”, “P-23”, “P-9”).	17
SECTION 15 — QA Architect Scenario Questions (Real-Time System Design)	17
Q16. It’s Friday, 5 PM. Your 800-test suite has been running 10 hours and still hasn’t finished — release is blocked. Walk through exactly what you do in the next 2 hours vs. the next 2 weeks.	17
Q17. A client-facing healthcare analytics platform needs a new tenant onboarded next week, with their own SSO provider, their own data isolation requirements, and a tight deadline. How do you scope the testing without either under-covering security or blowing the timeline?	18
Q18. Your team wants to migrate a 500-test Cypress suite to Playwright but can’t stop feature delivery for months to do it. Propose the migration plan.	19
Q19. Two feature teams both claim ownership of a shared checkout/report-generation flow, and their tests keep breaking each other’s PRs by testing overlapping behavior differently. How do you resolve this architecturally?	19
SECTION 16 — Scenario-Based Debugging (Broken Code, Live)	20
Q20. This test passes locally but fails in CI. Find the bug.	20
Q21. This test is meant to verify that only <i>active</i> cohorts appear in the list, but it’s passing even when the filter logic is broken. Find the bug in the test itself. . . .	21
Q22. Two tests in the same file conflict — running the file alone, both pass; running the full suite, the second one fails. Diagnose from the code alone.	22
Q23. This locator strategy breaks every time the design team ships a CSS refactor, even though the feature’s behavior never changed. Fix the underlying fragility. .	23

How to Use This Volume

This is **Volume 2** - the practical, real-time-facing companion to Volume 1 (Architecture, Fixtures, Auth, Locators, Sync, Parallelism, Config, API, Network Mocking, Debugging). Every question here is built around an actual production scenario, a real incident pattern, or working-but-broken code you'd genuinely be handed in an interview - not definitions or theory. Each answer states what actually happened, the trade-off considered, the fix applied, and how you'd defend that decision to an interviewer pushing back.

Frequency legend: ★★★★★ Very Frequently Asked · ★★★★ Frequently Asked · ★★★ Occasionally Asked · ★★ Rare but High-Value

SECTION 11 — Enterprise Framework & POM (Real Scenarios)

Q1. Your team inherited a 600-test Playwright suite where every Page Object method also contains expect() calls. A new hire's PR adds a test that fails with a confusing stack trace three layers deep inside a Page Object. How do you fix the architecture, not just this one test?

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, EPAM, Deloitte Digital **Frequency:** ★★★★★

Why Interviewer Asks This Almost every legacy suite has this exact disease. The interviewer wants to see if you can diagnose an architectural smell from a symptom (confusing stack trace) rather than just patching the one failing test.

Best Interview Answer The root cause is assertions living inside Page Objects instead of test files — when a expect() buried in CohortTable.assertRowCount() fails, the stack trace points into framework code, not the test's intent, and it's unclear from the test file alone what was actually being verified. I refactor incrementally, not in one big-bang PR: for each Page Object, I convert assertion methods into plain getters that expose state (getRowCount(): Promise<number>), move the expect() call into the calling test, and run the full suite after each file to confirm no regression. I don't do this suite-wide in one PR — legacy suites at this size need file-by-file migration with review, or the diff is unreviewable and risky.

Real Project Example A DashboardPage.verifyCohortLoaded() method did three internal assertions and returned nothing; migrating it to getCohortStatus(): Promise<'loading' | 'loaded' | 'error'> let the test itself decide what to assert and why, and the next failure's stack trace pointed directly at the test line, not three method calls deep.

TypeScript Code Example

```
// BEFORE — assertion buried in Page Object, confusing stack trace on failure
class CohortTable {
  async assertRowCount(expected: number) {
    const rows = await this.page.getByTestId('cohort-row').count();
    expect(rows).toBe(expected); // failure points here, not at the test's intent
  }
}

// AFTER — Page Object exposes state, test asserts
class CohortTable {
  async getRowCount(): Promise<number> {
    return this.page.getByTestId('cohort-row').count();
  }
}

test('cohort table shows filtered results', async ({ cohortTable }) => {
  expect(await cohortTable.getRowCount()).toBe(12); // failure points here, clear intent
});
```

Common Mistakes Attempting a suite-wide rewrite in one massive PR instead of an incremental, reviewable migration; refactoring without running the full suite after each change, silently breaking coverage.

Follow-Up Questions 1. How would you prioritize which Page Objects to migrate first in a 600-test suite? 2. How do you prevent new PRs from reintroducing assertions into Page Objects

going forward? 3. What lint rule or code-review checklist item would you add to enforce this?

QA Architect Tip Mention adding a custom ESLint rule or a PR template checklist item (“no expect() inside /pages/”) — architects don’t just fix the bug, they make the class of bug structurally harder to reintroduce.

Q2. Four engineers are now writing Playwright tests in parallel on the same feature area, and every PR has merge conflicts in one giant DashboardPage.ts file. How do you restructure this without blocking the team for a week?

Difficulty: Staff / QA Architect **Asked In:** Microsoft, ServiceNow, Atlassian **Frequency:** ★★★★★

Why Interviewer Asks This This is a team-scaling problem disguised as an architecture problem — exactly what an Associate Architect / QE Lead is expected to solve without halting delivery.

Best Interview Answer The fix is splitting the god Page Object into Component Objects by UI region — FilterPanel, CohortTable, ExportModal — each in its own file, each owned conceptually by whichever feature area touches it most. I do this as a single, mechanical, low-risk refactor PR (move methods to new files, no behavior change) merged quickly while feature work is paused for a few hours, then re-open feature branches on top of the new structure. Going forward, conflicts drop because engineers touching the filter panel and engineers touching the export modal are editing different files.

Real Project Example Splitting DashboardPage.ts (1,400 lines, 40+ methods) into 6 component files matching the actual UI regions cut merge-conflict-related PR delays from roughly 2-3 per week to near zero over the following month.

TypeScript Code Example

```
// Composition: DashboardPage now delegates to focused component objects
class DashboardPage {
  readonly filterPanel: FilterPanel;
  readonly cohortTable: CohortTable;
  readonly exportModal: ExportModal;

  constructor(page: Page) {
    this.filterPanel = new FilterPanel(page);
    this.cohortTable = new CohortTable(page);
    this.exportModal = new ExportModal(page);
  }
}

// tests/dashboards/export.spec.ts
test('export respects active filters', async ({ dashboardPage }) => {
  await dashboardPage.filterPanel.setAgeRange('40-60');
  await dashboardPage.exportModal.open();
  await dashboardPage.exportModal.export('csv');
});
```

Common Mistakes Doing the split as part of a feature PR (mixing structural refactor with behavior change), making the diff impossible to review safely.

Follow-Up Questions 1. How do you communicate this refactor to the team to minimize disruption to in-flight branches? 2. How would you decide the boundary lines between component objects when the UI itself is tightly coupled?

QA Architect Tip Explicitly separate “mechanical refactor PR” from “feature PR” — mixing them is the single most common reason big refactors get rejected or cause incidents.

Q3. A Page Object method works fine for the web app, but the mobile-responsive layout renders the same feature with a completely different DOM structure. Do you fork the Page Object, or is there a better pattern?

Difficulty: Senior **Asked In:** Walmart Global Tech, Flipkart, Meesho **Frequency:** ★★★★★

Best Interview Answer Forking into `DashboardPageDesktop` and `DashboardPageMobile` duplicates logic and doubles maintenance. I prefer a single Page Object with an internal strategy that resolves the correct locator based on viewport/breakpoint at runtime, or — cleaner — a shared interface with two thin implementations only for the genuinely different bits, while everything else (API calls, data setup, shared assertions) stays common. The decision depends on how much actually differs: if it’s one or two locators, an internal branch is fine; if the whole interaction model differs (a hamburger menu instead of a full nav bar), separate small implementation classes behind a shared interface is cleaner.

Real Project Example A cohort filter panel renders as a sidebar on desktop and a slide-up drawer on mobile — one `FilterPanel` class exposes `open()` and `setAgeRange()` publicly, with an internal `isMobileLayout()` check branching only the two or three methods that actually differ in interaction mechanics, while filter-value-setting logic stays fully shared.

TypeScript Code Example

```
class FilterPanel {
  constructor(private page: Page) {}

  async open(): Promise<void> {
    const isMobile = (await this.page.viewportSize()).width < 768;
    if (isMobile) {
      await this.page.getByRole('button', { name: 'Filters' }).click(); // opens drawer
    } else {
      // desktop sidebar is always visible, nothing to open
    }
  }

  async setAgeRange(range: string): Promise<void> {
    await this.page.getLabel('Age range').fill(range); // same on both layouts
  }
}
```

Common Mistakes Forking the entire Page Object for a difference that’s really confined to one or two interactions, doubling the maintenance surface for no real benefit.

Follow-Up Questions 1. At what point does the branching complexity justify actually splitting into separate classes? 2. How would you structure Playwright projects to run the same spec files against both desktop and mobile viewports?

QA Architect Tip Frame the decision rule explicitly: “shared interface, thin implementations

only where interaction mechanics genuinely diverge” — this is a general pattern, not mobile-specific.

Q4. Your test-data folder has grown into 80 loosely-related JSON files, and nobody’s sure which ones are still used. How do you clean this up without breaking the suite?

Difficulty: Senior **Asked In:** EPAM, Optum, Razorpay **Frequency:** ★★★

Best Interview Answer I don’t manually guess — I grep the codebase for references to each file path across tests/ and fixtures/, generating a used/unused list programmatically rather than by memory. Unused files get deleted in a dedicated cleanup PR (not mixed with feature work), and going forward, I push toward generating test data at runtime via the API client for anything that doesn’t genuinely need to be a static fixture — static JSON files should be reserved for data that’s expensive or impossible to generate on the fly (large realistic datasets, specific edge-case payloads).

Real Project Example A grep-based audit script found 31 of 80 JSON files with zero references anywhere in the codebase — leftover from three different past initiatives — and deleting them (after one team confirmation round) cut the folder to a maintainable size with clear ownership.

TypeScript Code Example

```
# Quick audit: find test-data files never referenced anywhere
for f in test-data/*.json; do
  name=$(basename "$f")
  count=$(grep -rL "$name" tests/ fixtures/ | wc -l)
  [ "$count" -eq 0 ] && echo "UNUSED: $f"
done
```

Common Mistakes Deleting files based on “I don’t remember using this” instead of an actual reference audit — risks silently breaking a test that references it indirectly through a shared helper.

Follow-Up Questions 1. How would you prevent this sprawl from recurring as the team grows? 2. When is a static fixture file actually the right choice over API-generated data?

QA Architect Tip Suggest a lightweight CI check that flags newly added test-data files with zero references after N days — cheap governance that prevents the exact mess being cleaned up.

SECTION 12 — CI/CD (Real Pipeline Incidents)

Q5. Your GitHub Actions pipeline passed on your feature branch but fails on main after merge, with no code difference in the failing test. What’s your triage process?

Difficulty: Senior / Staff **Asked In:** Microsoft, Walmart Global Tech, JP Morgan **Frequency:** ★★★★★

Why Interviewer Asks This This is a real, common, genuinely confusing incident — the interviewer wants a structured triage process, not a guess.

Best Interview Answer First, I check what’s actually different between the two runs beyond the code: did another PR merge to main in between that changed shared test data, a shared staging environment, or a dependency version lock file? Second, I check for environment/secrets drift — branch-protected environments sometimes have different secrets or environment variables than the PR-check environment. Third, I check for concurrency: if main’s pipeline runs against a shared staging environment that PR checks don’t touch, a different in-flight process (another team’s deploy, a scheduled job) could be the actual cause. I reproduce by re-running the exact same commit on main in isolation before assuming it’s an environment issue rather than a real regression.

Real Project Example A “no code difference” failure on main traced to a separate scheduled nightly data-refresh job that had started truncating a shared staging table at the exact time the main pipeline ran — a timing collision invisible from the PR branch’s isolated test data, not a code regression at all.

TypeScript Code Example

```
# Diagnostic step added temporarily to the main pipeline to capture more context
- name: Capture environment state for triage
  if: failure()
  run: |
    echo "Git SHA: ${github.sha}"
    echo "Triggered by: ${github.event_name}"
    curl -s https://staging.example.com/api/health | tee health-snapshot.json
```

Common Mistakes Assuming “no code diff” means “must be flaky” and adding a retry, when a genuine environment or timing collision is the real cause and will recur.

Follow-Up Questions 1. How would you set up monitoring to catch this class of issue before it causes a failed deploy gate? 2. How do you decide whether main’s pipeline should run against an isolated environment instead of shared staging?

QA Architect Tip The senior answer treats “environment collision” as equally likely as “code regression” from the start — junior answers jump straight to blaming the test.

Q6. Your 800-test suite takes 45 minutes in CI, and the team wants PR feedback under 10 minutes. Walk through exactly how you’d get there.

Difficulty: Staff / QA Architect **Asked In:** Walmart Global Tech, ServiceNow, Meesho **Frequency:** ★★★★★

Best Interview Answer I attack this in layers, measuring after each change rather than guessing which lever matters most. First, tag tests by risk/area and run only a fast, high-confidence smoke subset (maybe 150 tests covering critical paths) on every PR, with full regression gated to merge-to-main and nightly — this alone often gets PR feedback under 10 minutes. Second, for whatever subset does run on PRs, maximize actual parallelism: `fullyParallel: true`, workers matched to the runner’s real core count, and sharding across multiple parallel CI jobs. Third, audit setup cost — API-driven auth and data seeding instead of UI-driven, eliminating any `waitForTimeout()` calls, and caching browser binaries so `npx playwright install` isn’t re-downloading every run.

Real Project Example On an 800-test suite: tagging ~180 tests as @smoke and running only those on PRs (4 shards × 4 workers) got PR feedback to 6 minutes; full 800-test regression still

runs on merge and nightly at the original ~45 minutes, which is an acceptable trade-off since it's not blocking active development.

TypeScript Code Example

```
// Tag-based selective execution
test('cohort filter applies correctly', { tag: '@smoke' }, async ({ page }) => { /* ... */

# PR pipeline – fast subset only
npx playwright test --grep @smoke --shard=1/4

# Merge/nightly pipeline – full regression
npx playwright test --shard=1/4
```

Common Mistakes Chasing parallelism tuning first without first asking whether every test genuinely needs to run on every PR — the biggest win is usually scope reduction, not raw speed.

Follow-Up Questions 1. How do you decide which tests earn the @smoke tag, and who owns that decision? 2. What's your rollback plan if a bug ships because it was only caught by a test outside the PR-time smoke subset?

QA Architect Tip Explicitly rank the levers by expected impact before touching config — “scope reduction first, then parallelism, then setup-cost cleanup” — this ordering is what interviewers are actually listening for.

Q7. A flaky test has a 5% failure rate and is now auto-retried twice in CI, effectively hiding the flakiness from the team. How do you surface and fix it without just removing the retry and blocking every PR?

Difficulty: Senior / Staff **Asked In:** Salesforce, Atlassian, PhonePe **Frequency:** ★★★★★

Best Interview Answer I don't remove retries wholesale — that would tank team velocity while the real fix is in progress. Instead, I add flaky-test tracking: Playwright's HTML/JSON reporter data (or a CI plugin) can log which tests passed only after a retry, feeding a dashboard of “tests that retried this week” — this makes invisible flakiness visible without blocking anyone. I then prioritize fixing the highest-frequency offenders first, using trace data from the retry-triggering failures (traces are still captured on on-first-retry), and only remove that specific test's retry allowance once it's demonstrated stable for a set period.

Real Project Example A flaky-test dashboard surfaced that one specific export test was retrying on roughly 1 in 20 runs; tracing it to a business-readiness gap (Section 5, Q25 pattern) and fixing the actual wait condition dropped it to zero retries over the following two weeks, at which point it was removed from a “known flaky” watch-list.

TypeScript Code Example

```
export default defineConfig({
  retries: process.env.CI ? 2 : 0,
  reporter: [
    ['html'],
    ['json', { outputFile: 'test-results/results.json' }], // feeds flaky-test dashboard
  ],
});
```

```
# Extract tests that only passed after retry, from the JSON report
jq '.suites[].specs[] | select(.tests[].results | length > 1) | .title' test-results/resul
```

Common Mistakes Treating “retries are configured” as “the problem is handled” — retries buy time, they don’t fix anything, and left unmonitored they let real flakiness accumulate invisibly.

Follow-Up Questions 1. How would you set a policy for how long a test is allowed to stay on the “known flaky, being fixed” list before it’s quarantined or deleted? 2. How do you distinguish test flakiness from genuine intermittent app bugs using this same data?

QA Architect Tip Say explicitly that retries without tracking are “flakiness debt” — architects manage it like technical debt, with visibility and a paydown plan, not silence.

Q8. Your Docker-based Playwright pipeline works on your laptop’s Docker but fails with browser launch errors only inside the CI runner’s container. Diagnose it.

Difficulty: Senior **Asked In:** Microsoft, EPAM, Adobe **Frequency:** ★★★★★

Best Interview Answer Almost always a mismatch between the Playwright npm package version and the browser binaries baked into the Docker image, or missing OS-level dependencies (fonts, shared libraries) that the base image lacks but a full desktop OS has by default. I check `npx playwright --version` against the image tag (Microsoft’s official images are tagged per exact Playwright version — a mismatch here is the single most common cause), and I check the actual launch error for a missing shared library, which usually means the base image needs `npx playwright install-deps` run explicitly, not just `install`.

Real Project Example A pipeline using `mcr.microsoft.com/playwright:v1.44.0` while `package.json` had been bumped to Playwright 1.47.0 without updating the Docker image tag — browsers launched fine locally (host OS had all deps) but failed in the container with a missing-library error until the image tag was synced to match.

TypeScript Code Example

```
# Pin the Docker image tag to exactly match package.json's Playwright version
FROM mcr.microsoft.com/playwright:v1.47.0-noble
```

```
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
CMD ["npx", "playwright", "test"]
```

Common Mistakes Bumping the `@playwright/test` npm version without also bumping the Docker base image tag — this drift is silent until a browser launch actually fails.

Follow-Up Questions 1. How would you automate keeping the Docker image tag in sync with the npm package version? 2. What’s the fallback if you’re not using Microsoft’s official image at all?

QA Architect Tip Recommend a CI step that fails fast with a clear message if `npx playwright --version` doesn’t match the expected pinned version — turns a confusing runtime error into an obvious, actionable one.

SECTION 13 — Performance, Stability & Optimization (Real Tuning Decisions)

Q9. After adding more CI workers, your suite got slower, not faster. Explain what’s actually happening and how you’d right-size it.

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, JP Morgan, Razorpay **Frequency:** ★★★★★

Best Interview Answer Browser automation is CPU- and memory-bound — each worker runs its own browser process, and once worker count exceeds the runner’s actual CPU core count, workers start contending for the same cores, causing context switching overhead that makes everything slower, not faster. I right-size by checking the CI runner’s actual allocated cores (not assumed — many CI providers under-allocate relative to what `nproc` reports inside a container) and setting workers at or slightly below that number, then measuring wall-clock time empirically at a few different worker counts rather than assuming higher is always better.

Real Project Example A CI runner reporting 8 vCPUs but actually throttled to 4 real cores under the CI provider’s shared-tenancy plan: running 8 Playwright workers was measurably slower (12 minutes) than running 4 workers (7 minutes) on the exact same suite, because the extra workers were fighting for the same underlying compute.

TypeScript Code Example

```
export default defineConfig({
  workers: process.env.CI ? 4 : undefined, // matches actual CI core allocation, not the r
});
```

```
# Measure empirically rather than assuming
for w in 2 4 8 16; do
  echo "workers=$w:"
  time npx playwright test --workers=$w
done
```

Common Mistakes Treating “more workers = more parallelism = always faster” as a universal rule without ever measuring against the actual hardware constraint.

Follow-Up Questions 1. How would this calculation change on a memory-constrained runner rather than a CPU-constrained one? 2. How do you communicate this trade-off to a manager who just wants “make it faster” without understanding the mechanism?

QA Architect Tip Say plainly: “I don’t guess at worker count, I measure it empirically against the actual runner” — this measured approach is what separates a real optimization from a cargo-culted config change.

Q10. Your suite’s memory usage climbs steadily over a 3-hour nightly run and eventually OOM-kills the CI job partway through. Find the leak.

Difficulty: Senior / Staff **Asked In:** Microsoft, ServiceNow, Optum **Frequency:** ★★★★★

Best Interview Answer A steadily climbing (not sawtoothed) memory baseline across hundreds of tests within the same worker process points to something not being released between tests — usually unclosed `BrowserContexts` or pages from a fixture that opens extra contexts (like the multi-user test pattern) without closing them in a `finally`, or `page.route()` handlers

registered per-test that accumulate because they were never explicitly removed. I'd instrument by logging active context/page counts periodically during a local reproduction run, and audit every fixture that creates a context or page for a guaranteed matching close in teardown.

Real Project Example A multi-user collaboration test fixture (Section 1, Q7 pattern) opened two extra contexts per test but only closed them on the happy path — any test in that file that failed an assertion before reaching the cleanup line leaked two contexts per failure, and across a 3-hour run with a handful of failures, that was enough to exhaust memory by hour two.

TypeScript Code Example

```
// LEAK: contexts only closed if the test body doesn't throw first
test('collaboration test', async ({ browser }) => {
  const contextA = await browser.newContext();
  const contextB = await browser.newContext();
  // ... test body throws here on failure, contexts never close
  await contextA.close();
  await contextB.close();
});

// FIXED: fixture guarantees cleanup regardless of test outcome
export const test = base.extend<{ secondContext: BrowserContext }>({
  secondContext: async ({ browser }, use) => {
    const context = await browser.newContext();
    try {
      await use(context);
    } finally {
      await context.close(); // always runs
    }
  },
});
```

Common Mistakes Manually opening extra contexts/pages inside test bodies instead of through fixtures — fixtures give guaranteed teardown via try/finally, manual code in the test body does not unless you write it explicitly every time.

Follow-Up Questions 1. How would you confirm the leak is contexts/pages specifically vs. something else like accumulated console log listeners? 2. How would you set up alerting to catch memory creep before it causes an OOM kill in production CI?

QA Architect Tip Point out that this is exactly why extra-context patterns belong in fixtures, not raw test-body code — Q10 and the fixture-teardown discussion in Volume 1 (Section 2, Q12) are the same underlying discipline applied at different scales.

Q11. A specific test suite runs fine in isolation but consistently times out only when run as the last file in a full 800-test run. What's your hypothesis?

Difficulty: Staff / QA Architect **Asked In:** Salesforce, Deloitte Digital, Flipkart **Frequency:** ★★★

Best Interview Answer “Only fails at the end of a long run” points to resource exhaustion accumulating over time rather than a bug in that specific test — likely candidates are a slow memory leak (Q10's pattern) finally tipping over near the end, file-handle or network-socket exhaustion from something not being closed per-test, or a shared external resource (a rate

limit, a connection pool) that's been slowly consumed across the whole run and finally runs dry. I'd check system-level metrics (memory, open file descriptors, open network connections) over the course of a full local reproduction run, correlated against test count, rather than looking at the failing test's code first — the failing test is very likely a victim, not the cause.

Real Project Example A database connection pool fixture, worker-scoped and correctly built, still leaked one connection per test because a specific query path didn't release it on a particular response shape; by test 750 of 800, the pool was exhausted and the last handful of tests timed out waiting for a connection that would never free up.

TypeScript Code Example

```
// Diagnostic: log resource counts periodically during a local repro run
test.afterEach(async () => {
  if (process.env.DEBUG_RESOURCES) {
    console.log('Open handles:', process.getActiveResourcesInfo?().length);
  }
});
```

Common Mistakes Debugging the last-failing test's own logic in isolation, when in isolation it will almost always pass — the bug isn't in that test, it's in the cumulative resource cost of everything before it.

Follow-Up Questions 1. How would you narrow down which earlier test(s) are the actual source of the leak? 2. How do you build a regression test specifically for “does this fixture leak resources over N repeated runs”?

QA Architect Tip The strongest signal here is immediately reframing the question from “why does this test fail” to “why did the system run out of some resource by this point” — that reframe is what Staff-level interviewers are listening for.

SECTION 14 — Real Coding Interview Questions (Live Coding Style)

Q12. Build a fixture that logs in as one of three roles (admin, analyst, auditor), reusing cached auth per worker, and expose it as test.use({ role: 'admin' }) per describe block.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, JP Morgan **Frequency:** ★★★★★

Why Interviewer Asks This This is the single most commonly assigned live-coding exercise in 2025–2026 Playwright interviews because it combines fixtures, scope, and auth architecture in one exercise that's gradeable in 15 minutes.

Best Interview Answer (spoken while coding) “I'll make role a worker-scoped option fixture defaulting to analyst, and a storageState fixture that resolves to a pre-generated file per role — the actual logins happen once in globalSetup, not inside this fixture, so I'm not re-authenticating per test or per worker.”

TypeScript Code Example

```
// global-setup.ts – runs once, before any worker starts
import { request } from '@playwright/test';

const roles = ['admin', 'analyst', 'auditor'] as const;
```

```

export default async function globalSetup() {
  for (const role of roles) {
    const context = await request.newContext();
    await context.post('/api/auth/login', {
      data: { username: process.env[`${role.toUpperCase()}_USER`], password: process.env[
    });
    await context.storageState({ path: `.auth/${role}.json` });
    await context.dispose();
  }
}

```

```

// fixtures/role.ts
import { test as base } from '@playwright/test';

type Role = 'admin' | 'analyst' | 'auditor';

export const test = base.extend<{}, { role: Role }>({
  role: ['analyst', { option: true, scope: 'worker' }],

  storageState: [async ({ role }, use) => {
    await use(`.auth/${role}.json`);
  }, { scope: 'worker' }],
});

export { expect } from '@playwright/test';

```

```

// tests/admin/user-management.spec.ts
import { test, expect } from '../../fixtures/role';

test.describe('admin-only actions', () => {
  test.use({ role: 'admin' });

  test('admin can deactivate a user', async ({ page }) => {
    await page.goto('/admin/users');
    await page.getByRole('row', { name: 'Jane Doe' }).getByRole('button', { name: 'Deactiv
    await expect(page.getByText('User deactivated')).toBeVisible();
  });
});

```

Common Mistakes Making storageState test-scoped instead of worker-scoped, causing it to be re-resolved (harmless here, but wasteful) or, worse, putting the actual login call inside the fixture, re-authenticating on every worker instead of once total.

Follow-Up Questions 1. How would you extend this to support a role parameterized per individual test rather than per describe block? 2. What happens if ADMIN_USER env var is missing — how would you fail fast with a clear error?

QA Architect Tip Narrate the “acquire once, consume many times” separation explicitly while coding — interviewers are listening for that phrase as much as the working code.

Q13. This test intermittently fails with “element is not attached to the DOM.” Find the bug and fix it.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, Swiggy **Frequency:** ★★★★★

Given (broken) code:

```
test('applies filter and shows updated count', async ({ page }) => {
  await page.goto('/dashboards/cohort-explorer');
  const countLocator = page.getByTestId('cohort-count');
  const initialText = await countLocator.textContent();

  await page.getByLabel('Age range').fill('40-60');
  await page.getByRole('button', { name: 'Apply' }).click();

  await page.waitForTimeout(1000);
  expect(await countLocator.textContent()).not.toBe(initialText);
});
```

Why Interviewer Asks This This is a realistic bug: the app re-renders the count element (replacing the DOM node) after filtering, and the cached countLocator reference — actually fine, since Playwright locators are lazy and re-query each time — is a red herring; the *real* bug is the fixed waitForTimeout racing the re-render, occasionally reading the old text before the new render completes, or occasionally after the element has been detached and re-attached mid-check.

Best Interview Answer Locators in Playwright are lazy — countLocator re-queries the DOM every time you call an action on it, so it’s not actually stale by itself. The real problem is wait-ForTimeout(1000) racing an unbounded re-render: under load, 1000ms may not be enough, and even when it is, textContent() called at exactly the wrong instant can catch the DOM mid-swap. The fix is a web-first expect().not.toHaveText() which auto-retries until the assertion passes or times out, rather than a single point-in-time read after an arbitrary sleep.

Fixed Code:

```
test('applies filter and shows updated count', async ({ page }) => {
  await page.goto('/dashboards/cohort-explorer');
  const countLocator = page.getByTestId('cohort-count');
  const initialText = await countLocator.textContent();

  await page.getByLabel('Age range').fill('40-60');
  await page.getByRole('button', { name: 'Apply' }).click();

  await expect(countLocator).not.toHaveText(initialText ?? '');
});
```

Common Mistakes Diagnosing this as a “stale locator” problem and trying to re-fetch the locator after the click — that’s not the actual bug, since Playwright locators don’t go stale the way raw WebElement references do.

Follow-Up Questions 1. Why don’t Playwright locators go “stale” the way Selenium’s WebElement references do? 2. How would you assert on the *new* value specifically, not just “not the old value”?

QA Architect Tip Say explicitly why the locator itself isn’t the bug — showing you understand Playwright’s lazy-locator model rather than pattern-matching to a Selenium-era diagnosis is the actual signal here.

Q14. Automate uploading a file via a custom drag-and-drop zone that doesn't respond to `setInputFiles()` because there's no visible `<input type="file">`.

Difficulty: Senior **Asked In:** Walmart Global Tech, Adobe, Flipkart **Frequency:** ★★★★★

Best Interview Answer Even a custom drag-and-drop zone almost always has a hidden `<input type="file">` behind the scenes — browsers require it for the underlying file-picker API to work at all, it's just visually hidden or positioned off-screen. I locate that hidden input directly (even if `display: none`, Playwright's `setInputFiles()` works on hidden inputs specifically, unlike `click()` which needs visibility) rather than trying to simulate real OS-level drag events, which is far more brittle.

TypeScript Code Example

```
test('uploads CSV via custom drop zone', async ({ page }) => {
  await page.goto('/dashboards/import');

  // The hidden input behind the visual drop zone
  const fileInput = page.locator('input[type="file"]');
  await fileInput.setInputFiles('test-data/valid-cohort.csv');

  await expect(page.getByText('Upload successful')).toBeVisible();
});
```

If genuinely no hidden input exists (rare, fully custom JS drag implementation):

```
test('uploads via simulated DataTransfer drop event', async ({ page }) => {
  const dropZone = page.getByTestId('drop-zone');
  const buffer = await fs.promises.readFile('test-data/valid-cohort.csv');

  const dataTransfer = await page.evaluateHandle((data) => {
    const dt = new DataTransfer();
    const file = new File([new Uint8Array(data)], 'valid-cohort.csv', { type: 'text/csv' });
    dt.items.add(file);
    return dt;
  }, Array.from(buffer));

  await dropZone.dispatchEvent('drop', { dataTransfer });
});
```

Common Mistakes Jumping straight to simulating raw mouse drag events (`mouse.down()/move()/up()`) for file upload specifically — that mechanism is for element-to-element drag-and-drop, not OS file drops, and won't trigger a real file selection.

Follow-Up Questions 1. How would you verify the upload actually succeeded beyond a success toast — e.g., checking the file's contents were processed correctly? 2. How would this differ for testing multiple simultaneous file drops?

QA Architect Tip Mention checking DevTools' Elements panel for a hidden input as your first move before writing any exotic drag-simulation code — most "impossible" upload widgets aren't.

Q15. Write a test that verifies a table’s data is correctly sorted after clicking a column header, for a column containing mixed numeric-looking strings (e.g., patient IDs like “P-102”, “P-23”, “P-9”).

Difficulty: Senior **Asked In:** ServiceNow, Optum, Meesho **Frequency:** ★★★★★

Best Interview Answer The trap here is that “P-102”, “P-23”, “P-9” sort differently depending on whether the comparison is lexicographic (string) or numeric-aware — a naive test asserting a specific expected order without first deciding which sort semantic the app actually implements will either pass by accident or fail for the wrong reason. I extract all values, compute what I expect sorted order to be given the app’s documented/intended sort behavior (usually numeric-aware for ID-like columns), and assert the actual rendered order matches that computed expectation — not a hardcoded literal list, which breaks the moment test data changes.

TypeScript Code Example

```
test('patient ID column sorts numerically, not lexicographically', async ({ page }) => {
  await page.goto('/dashboards/patients');
  await page.getByRole('columnheader', { name: 'Patient ID' }).click();

  const cells = await page.getByTestId('patient-id-cell').allTextContents();

  const extractNumber = (id: string) => parseInt(id.replace('P-', ''), 10);
  const expectedOrder = [...cells].sort((a, b) => extractNumber(a) - extractNumber(b));

  expect(cells).toEqual(expectedOrder);
});
```

Common Mistakes Hardcoding an expected literal array like ['P-9', 'P-23', 'P-102'] tied to today’s specific seeded data — the test breaks the instant test data changes, even though sorting logic itself is still correct.

Follow-Up Questions 1. How would you test the *descending* sort direction on the second click? 2. How would you test sort stability when two rows have equal sort keys?

QA Architect Tip Point out explicitly that the test should encode the *sorting rule*, not a *snapshot of today’s data* — this distinction between “testing behavior” vs. “testing a specific fixture” is a recurring theme across strong answers in this whole handbook.

SECTION 15 — QA Architect Scenario Questions (Real-Time System Design)

Q16. It’s Friday, 5 PM. Your 800-test suite has been running 10 hours and still hasn’t finished — release is blocked. Walk through exactly what you do in the next 2 hours vs. the next 2 weeks.

Difficulty: Staff / QA Architect **Asked In:** Walmart Global Tech, JP Morgan, ServiceNow **Frequency:** ★★★★★

Why Interviewer Asks This This tests incident response under real pressure, not idealized architecture — the “next 2 hours” answer and the “next 2 weeks” answer need to be genuinely different, or the candidate hasn’t actually managed a release-blocking incident before.

Best Interview Answer Next 2 hours: I don’t try to fix the architecture live. I check if the

run is actually hung (a deadlocked test) vs. genuinely slow — if hung, kill and identify the specific hanging test, quarantine it, and re-run everything else to unblock the release decision with partial-but-trustworthy coverage plus a manual smoke check on the quarantined area. If it's genuinely just slow, I check whether a subset covering the release's actual changed area has already passed, and make a risk-based call with the release owner about shipping on partial coverage versus waiting, rather than unilaterally deciding either way. Next 2 weeks: root-cause why 800 tests takes 10 hours at all — this is the exact problem in Section 12, Q6 (Volume 2) — smoke-tag a fast subset for future release gates, fix worker/shard sizing, and eliminate setup-cost bloat, so this incident can't recur.

Real Project Example A genuinely hung test (an infinite retry loop waiting on a WebSocket event that would never fire due to an unrelated staging outage) was blocking an entire release; killing that one test, confirming the rest of the suite passed, and shipping with a manual verification of just that one feature got the release out same-day, while the following sprint's work fixed the suite's actual runtime problem.

Common Mistakes Treating this as purely a technical problem and not also a stakeholder-communication problem — Staff-level candidates are expected to mention looping in the release owner on the risk trade-off, not just fixing code silently.

Follow-Up Questions 1. How do you decide the threshold for “safe enough to ship on partial coverage”? 2. How would you prevent a single hung test from blocking the entire suite's output in the future?

QA Architect Tip Explicitly split your answer into “stop the bleeding now” vs. “fix the disease later” — interviewers are checking whether you can operate under time pressure without pretending you'd redesign the whole framework in the next hour.

Q17. A client-facing healthcare analytics platform needs a new tenant onboarded next week, with their own SSO provider, their own data isolation requirements, and a tight deadline. How do you scope the testing without either under-covering security or blowing the timeline?

Difficulty: Staff / QA Architect **Asked In:** ServiceNow, Optum, Adobe **Frequency:** ★★★★★

Best Interview Answer I risk-stratify rather than trying to fully regression-test everything for the new tenant in a week. Highest priority, non-negotiable: cross-tenant data isolation (does this new tenant's users ever see another tenant's data) and SSO configuration correctness (does login actually work end-to-end with their specific IdP) — these get dedicated, thorough new tests because a failure here is a severe security incident, not a UX bug. Medium priority: tenant-specific configuration (branding, feature flags enabled for this client) gets targeted smoke coverage. Lowest priority: I explicitly do *not* re-run full functional regression assuming this tenant behaves identically to existing ones for already-proven-stable features — that's what the existing suite already covers, parameterized by tenant where the architecture (Section 3, Q18 in Volume 1) already supports it.

Real Project Example Onboarding a new health-system client onto an Artemis-Health-style platform: two full days went into cross-tenant isolation tests and Okta-specific SSO validation for their exact IdP config; the existing parameterized functional suite ran against the new tenant's storage state with zero new authoring needed, since the framework was already designed for multi-tenant parameterization.

Common Mistakes Spending the tight timeline on broad functional regression “just to be safe” while under-investing in the actually high-risk area (cross-tenant isolation), which is exactly

backwards from a risk-based allocation.

Follow-Up Questions 1. What would you do differently if the existing framework did *not* already support tenant parameterization? 2. How would you communicate to the client's stakeholders what was and wasn't tested before go-live, given the timeline?

QA Architect Tip State the risk-stratification explicitly and in that order (security/isolation first, config second, redundant regression last) — this ordering, spoken clearly, is the actual signal Staff interviewers score on.

Q18. Your team wants to migrate a 500-test Cypress suite to Playwright but can't stop feature delivery for months to do it. Propose the migration plan.

Difficulty: Staff / QA Architect **Asked In:** Microsoft, EPAM, Adobe **Frequency:** ★★★★★

Best Interview Answer I never do a big-bang rewrite. Both frameworks run side-by-side during migration — new tests are written in Playwright from day one, and existing Cypress tests are ported incrementally, prioritized by business-criticality and by which ones are already flaky/high-maintenance in Cypress (a natural opportunity to fix them during the rewrite rather than faithfully porting bugs). CI runs both suites in parallel jobs during the transition, with a tracked migration percentage visible to the team, and a hard cutover date only set once coverage parity is confirmed — not on a calendar guess made on day one.

Real Project Example Migrating 500 Cypress tests: prioritized the 80 tests covering revenue-critical checkout/report-generation flows in month one, ported the remaining tests by feature area over the following quarter at a sustainable pace alongside normal feature work, and only decommissioned the Cypress CI job once a coverage-parity checklist was fully signed off by feature owners.

Common Mistakes Setting an arbitrary “Cypress is dead in 6 weeks” deadline before establishing actual coverage parity — this either blows the deadline or ships with silent coverage gaps.

Follow-Up Questions 1. How do you handle Cypress-specific patterns (like `cy.intercept()`) that don't map 1:1 to Playwright's model? 2. How would you convince a skeptical engineer that porting is worth pausing some feature-test authoring for?

QA Architect Tip Emphasize using the migration as an opportunity to fix known-flaky tests rather than faithfully porting their flakiness — a purely mechanical port just moves the same technical debt into a new framework.

Q19. Two feature teams both claim ownership of a shared checkout/report-generation flow, and their tests keep breaking each other's PRs by testing overlapping behavior differently. How do you resolve this architecturally?

Difficulty: Staff / QA Architect **Asked In:** Salesforce, Atlassian, Razorpay **Frequency:** ★★★

Best Interview Answer This is an ownership boundary problem wearing a technical costume. I'd propose splitting the shared flow's test ownership explicitly by layer or by sub-feature rather than leaving it ambiguous — for example, team A owns the report-generation logic's tests, team B owns the export/delivery mechanism's tests, with a documented, narrow integration

test (owned jointly, changed only with both teams' review) covering just the handoff between the two. I'd also introduce a shared Page Object/component library for the overlapping UI so both teams' tests reference the same locators, preventing one team's markup refactor from silently breaking the other's tests through duplicated, drifted locator definitions.

Real Project Example A report-generation flow owned ambiguously by both a "dashboards" team and an "exports" team had duplicated Page Object classes with slightly different locators for the same "Generate Report" button — consolidating into one shared component object and drawing an explicit ownership line at the API boundary between generation and delivery stopped the recurring cross-team breakage.

Common Mistakes Trying to solve this purely with more communication/process without also fixing the underlying technical duplication (two Page Objects for the same UI) that's the actual mechanism causing repeated breakage.

Follow-Up Questions 1. How do you handle a genuine disagreement between the two teams about test coverage responsibility? 2. What CI/CODEOWNERS mechanism would you put in place to enforce the new boundary?

QA Architect Tip Connect this back to Conway's Law explicitly, as in Volume 1 Section 1 Q4 — architects are expected to recognize that recurring cross-team test conflicts are almost always an ownership-boundary problem first, a code problem second.

SECTION 16 — Scenario-Based Debugging (Broken Code, Live)

Q20. This test passes locally but fails in CI. Find the bug.

Difficulty: Senior **Asked In:** Microsoft, Walmart Global Tech, ServiceNow **Frequency:** ★★★★★

Given (broken) code:

```
test('exports report and shows success toast', async ({ page }) => {
  await page.goto('/dashboards/cohort/123');
  await page.getByRole('button', { name: 'Export' }).click();
  await page.waitForTimeout(2000);
  await expect(page.getByText('Export complete')).toBeVisible();
});
```

What's wrong / why flaky in CI The fixed 2-second sleep assumes the export completes within that window every time. Locally, on a fast machine with low contention, it usually does; in CI, under worker contention (Volume 2, Section 13, Q9) or genuine backend load, the export can take longer, and the assertion runs before "Export complete" ever appears — a classic actionability-vs-business-readiness gap (Volume 1, Section 5, Q25).

Fix

```
test('exports report and shows success toast', async ({ page }) => {
  await page.goto('/dashboards/cohort/123');
  await page.getByRole('button', { name: 'Export' }).click();
  await expect(page.getByText('Export complete')).toBeVisible({ timeout: 15000 });
});
```

Follow-Up Questions 1. How would you make this deterministic instead of just “generously timed”? 2. What if “Export complete” briefly flashes and then disappears — would this fix still work?

QA Architect Tip Note that a web-first expect isn’t just “a wait with a longer number” — it’s polling-based and returns the instant the condition is true, so it’s typically *faster* than a fixed sleep on the happy path, not just safer on the slow path.

Q21. This test is meant to verify that only *active* cohorts appear in the list, but it’s passing even when the filter logic is broken. Find the bug in the test itself.

Difficulty: Senior **Asked In:** EPAM, Optum, Deloitte Digital **Frequency:** ★★★★★

Given (broken) code:

```
test('only active cohorts are shown', async ({ page, apiClient }) => {
  await apiClient.createTestCohort({ name: 'Active One', status: 'active' });
  await apiClient.createTestCohort({ name: 'Archived One', status: 'archived' });

  await page.goto('/dashboards/cohorts?filter=active');

  const rows = page.getByTestId('cohort-row');
  await expect(rows).toHaveCount(1);
});
```

What’s wrong This assumes the test starts from a clean slate with exactly these two cohorts — but if the shared staging environment already has other active cohorts from other tests or seed data, `toHaveCount(1)` is either a false failure (too many) or, worse, a false pass if some other coincidence makes the count land on 1 anyway. The test is fragile because it asserts an absolute count in a shared environment instead of asserting the *specific* cohorts it created.

Fix

```
test('only active cohorts are shown, archived ones are excluded', async ({ page, apiClient }) => {
  const activeId = await apiClient.createTestCohort({ name: `Active-${Date.now()}`, status: 'active' });
  const archivedId = await apiClient.createTestCohort({ name: `Archived-${Date.now()}`, status: 'archived' });

  await page.goto('/dashboards/cohorts?filter=active');

  await expect(page.getByTestId(`cohort-row-${activeId}`)).toBeVisible();
  await expect(page.getByTestId(`cohort-row-${archivedId}`)).not.toBeVisible();

  await apiClient.deleteTestCohort(activeId);
  await apiClient.deleteTestCohort(archivedId);
});
```

Follow-Up Questions 1. Why is asserting presence/absence of specific known entities more robust than asserting a total count in a shared environment? 2. How would you adapt this if the UI truly needs a total-count assertion (e.g., a pagination test)?

QA Architect Tip This is the same “test behavior, not a snapshot of shared data” principle from Volume 2 Q15 — absolute counts in shared environments are one of the most common sources of both false failures and dangerous false passes.

Q22. Two tests in the same file conflict — running the file alone, both pass; running the full suite, the second one fails. Diagnose from the code alone.

Difficulty: Senior / Staff **Asked In:** Walmart Global Tech, JP Morgan, PhonePe **Frequency:** ★★★★★

Given (broken) code:

```
let cachedCohortId: string;

test('creates and views a cohort', async ({ page, apiClient }) => {
  cachedCohortId = await apiClient.createTestCohort({ name: 'Shared Test Cohort' });
  await page.goto(`/dashboards/cohort/${cachedCohortId}`);
  await expect(page.getByText('Shared Test Cohort')).toBeVisible();
});

test('renames a cohort', async ({ page, apiClient }) => {
  await apiClient.renameCohort(cachedCohortId, 'Renamed Cohort');
  await page.goto(`/dashboards/cohort/${cachedCohortId}`);
  await expect(page.getByText('Renamed Cohort')).toBeVisible();
});
```

What's wrong `cachedCohortId` is a module-level variable — under `fullyParallel` or even standard file-level parallelism with worker reuse across files, if these two tests happen to run in a different order than written, or if a *third*, unrelated test file sharing the same worker process resets/imports this module differently, the second test can run before the first has set `cachedCohortId`, or against a stale value. This is exactly the shared-state class of bug from Volume 1, Section 6, Q32 — module-level state persisting across tests in ways the test order isn't guaranteed to protect against.

Fix

```
test.describe.serial('cohort lifecycle', () => {
  let cohortId: string;

  test('creates and views a cohort', async ({ page, apiClient }) => {
    cohortId = await apiClient.createTestCohort({ name: 'Shared Test Cohort' });
    await page.goto(`/dashboards/cohort/${cohortId}`);
    await expect(page.getByText('Shared Test Cohort')).toBeVisible();
  });

  test('renames a cohort', async ({ page, apiClient }) => {
    await apiClient.renameCohort(cohortId, 'Renamed Cohort');
    await page.goto(`/dashboards/cohort/${cohortId}`);
    await expect(page.getByText('Renamed Cohort')).toBeVisible();
  });
});
```

Follow-Up Questions 1. Why does `describe.serial` fix the ordering guarantee that a plain module-level variable doesn't? 2. What's a fully independent alternative that avoids needing `serial` at all?

QA Architect Tip Point out that the *better* long-term fix is often making “renames a cohort” independently create its own cohort via `apiClient` rather than depending on a previous test's output at all — `serial` fixes the symptom safely, full independence removes the coupling entirely.

Q23. This locator strategy breaks every time the design team ships a CSS refactor, even though the feature’s behavior never changed. Fix the underlying fragility.

Difficulty: Medium / Senior **Asked In:** Every company in this handbook’s dataset **Frequency:** ★★★★★

Given (broken) code:

```
await page.locator('.MuiButton-root.css-1a2b3c4').click();
await page.locator('div.cohort-panel > div:nth-child(3) > span').textContent();
```

What’s wrong Both locators are coupled to implementation details that have nothing to do with the feature’s actual behavior — auto-generated CSS-in-JS class hashes (css-1a2b3c4) that change on every build, and positional DOM structure (nth-child(3)) that breaks the moment a designer adds or reorders an unrelated element. Neither reflects what a real user or assistive technology perceives.

Fix

```
await page.getByRole('button', { name: 'Export' }).click();
await page.getByTestId('cohort-member-count').textContent();
```

Follow-Up Questions 1. How would you convince a dev team resistant to adding data-testid attributes that this is worth the small effort? 2. What’s your fallback locator strategy for a third-party component you can’t modify at all?

QA Architect Tip Frame this as a cost conversation with the dev team: “every CSS refactor currently breaks N tests for zero functional reason — a five-minute data-testid addition per component eliminates that recurring tax permanently.” Architects sell resilient locators as an engineering-cost argument, not a QA preference.
